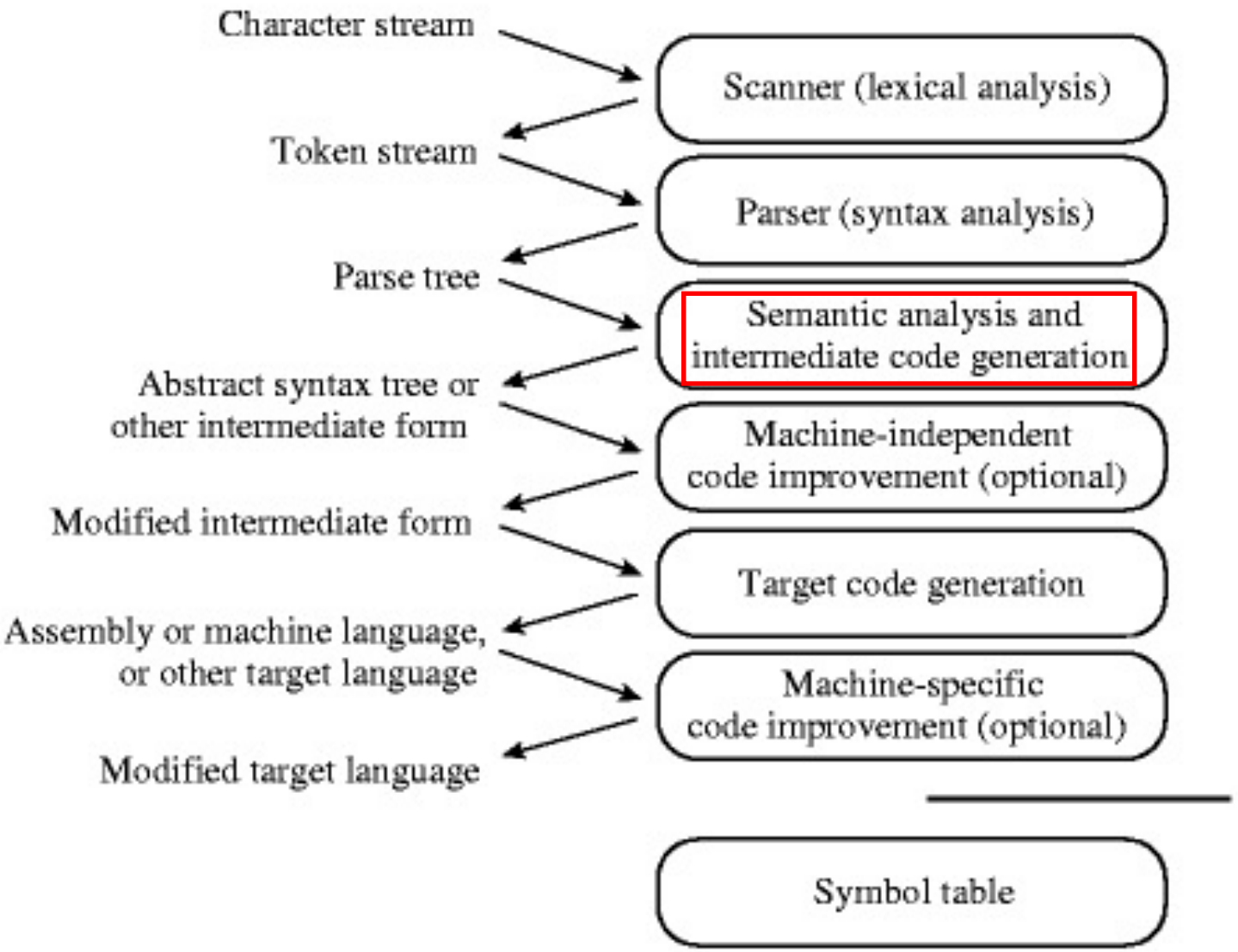
WEEK 3

SEMANTICS

Phases of Compilation



Semantic Analysis

- The principal job of the semantic analyzer is to enforce static semantic rules
  - constructs a syntax tree (usually first)
  - information gathered is needed by the code generator
- Static Semantics
  - Nothing to do with meaning
  - Context-free grammars (CFGs) cannot describe all of the syntax of programming languages
    - types of operands
    - variables declared before used

Attribute Grammars

- Attribute grammars (AGs) have additions to CFGs to carry some semantic info on parse tree nodes

- Primary value of AGs:
  - Static semantics specification
  - Compiler design (static semantics checking)

Attribute Grammars

- The attribute grammar serves to define the semantics of the input program
- Attribute rules are best thought of as definitions, not assignments
- They are not necessarily meant to be evaluated at any particular time, or in any particular order, though they do define their left-hand side in terms of the right-hand side

Attribute Grammars: An Example

- The variables can be one of two types: int or real
- When there are two variables on the right side of an assignment, they need not be the same type
- The type of the expression:
  - when the operand types are not the same: always real
  - when they are the same: the type of the operands
- The type of the left side of the assignment must match the type of the right side.
  - So the types of operands in the right side can be mixed, but the assignment is valid only if the target and the value resulting from evaluating the right side have the same type.

Attribute Grammars: An Example

- Syntax

```
<assign>  -> <var> = <expr>
<expr>    -> <var> + <var> | <var>
<var>     -> A | B | C
```

- `actual_type`:
  - synthesized for `<var>` and `<expr>`
- `expected_type`:
  - inherited for `<expr>`

Synthesized attributes: pass semantic information up a parse tree
Inherited attributes: pass semantic information down and across a tree.

Attribute Grammars: An Example

1. Syntax rule: <assign> → <var> = <expr>
   Semantic rule: <expr>.expected_type ← <var>.actual_type

2. Syntax rule: <expr> → <var>[2] + <var>[3]
   Semantic rule: <expr>.actual_type ←
   if (<var>[2].actual_type = int) and
   (<var>[3].actual_type = int)
   then int
   else real
   end if
   Predicate: <expr>.actual_type == <expr>.expected_type

Attribute Grammars: An Example

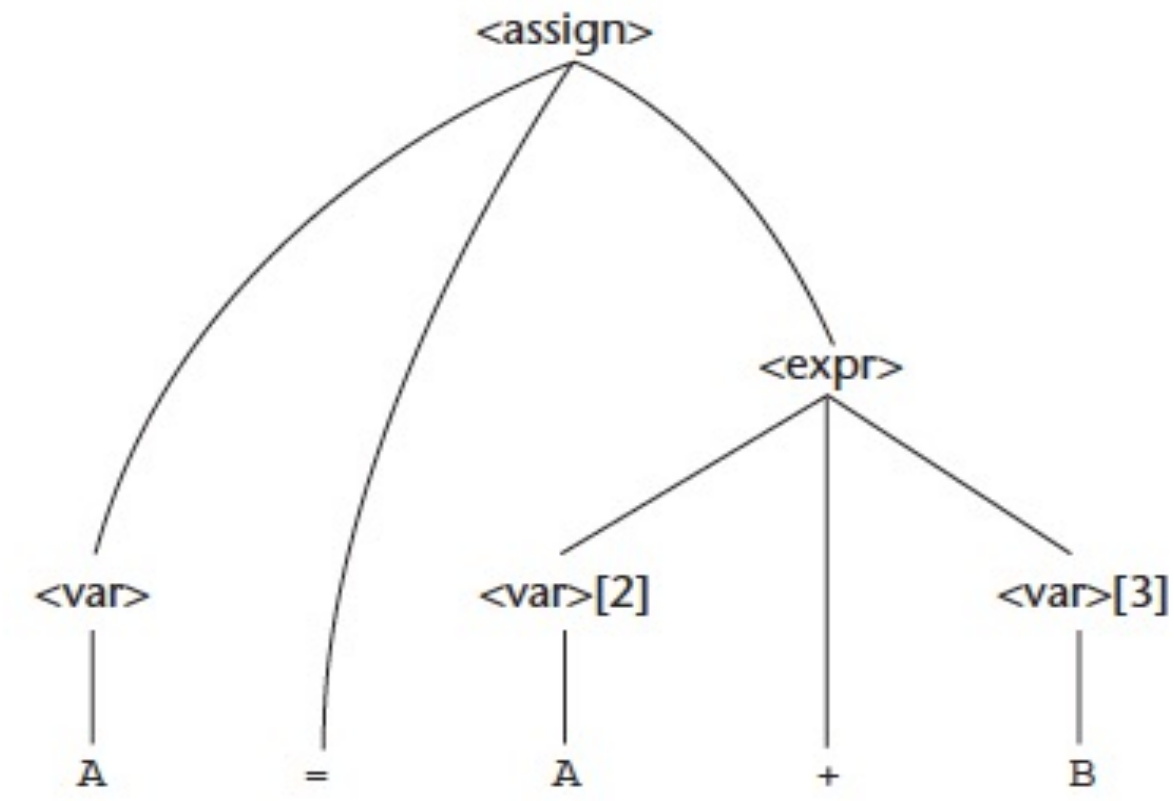
3. Syntax rule: <expr> → <var>
   Semantic rule: <expr>.actual_type ← <var>.actual_type
   Predicate: <expr>.actual_type == <expr>.expected_type

4. Syntax rule: <var> → A | B | C
   Semantic rule: <var>.actual_type ← look-up(<var>.string)

*** The look-up function looks up a given variable name in the symbol table and returns the variable's type.

Attribute Grammars: An Example

- A parse tree for A = A + B



Computing Attribute Values

- Sometimes called *decorating the parse tree*
- If all attributes were inherited, the tree could be decorated in top-down order.
- If all attributes were synthesized, the tree could be decorated in bottom-up order.
- In many cases, both kinds of attributes are used, and it is some combination of top-down and bottom-up that must be used.

Computing Attribute Values

1. <var>.actual_type ← look-up(A) (Rule 4)

2. <expr>.expected_type ← <var>.actual_type (Rule 1)

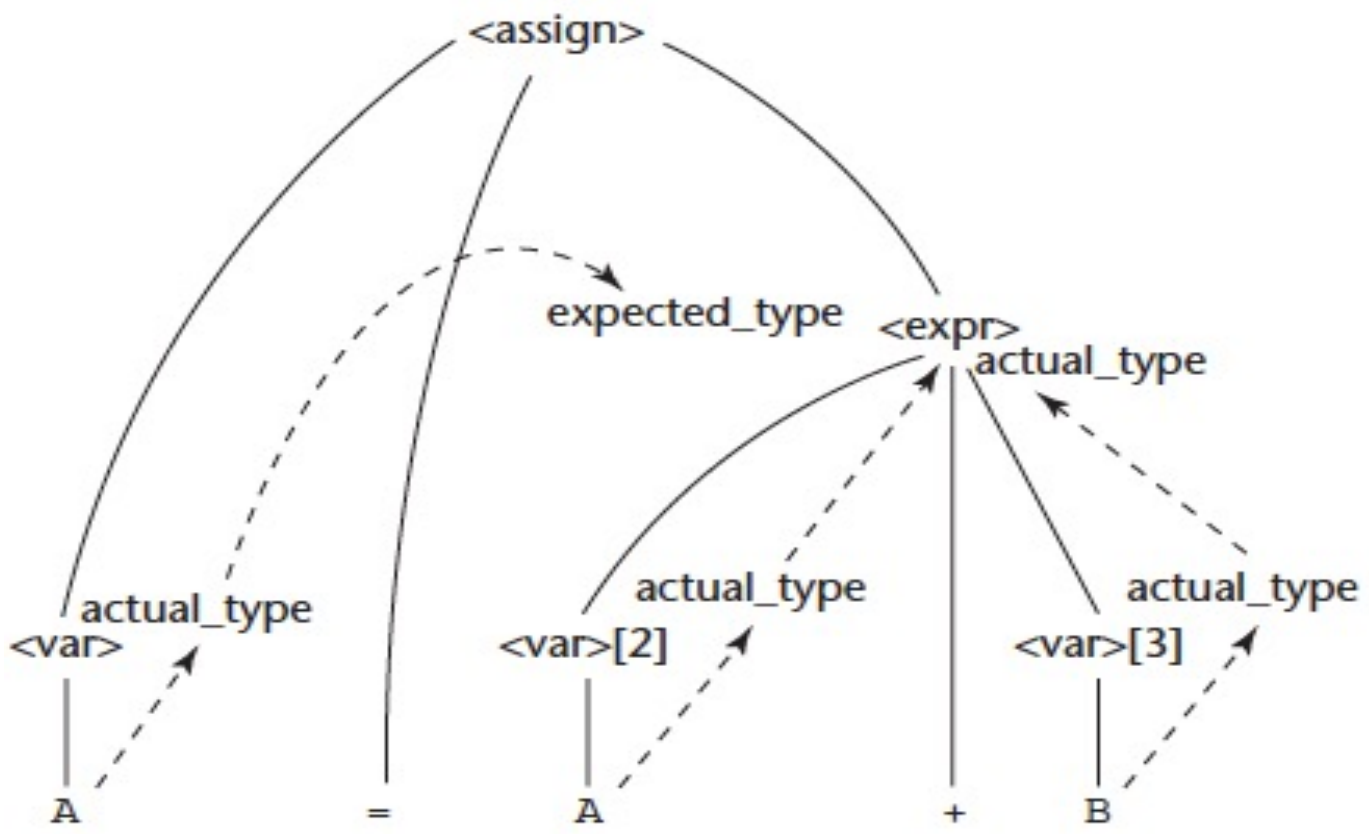
3. <var>[2].actual_type ← look-up (A) (Rule 4)
   <var>[3].actual_type ← look-up (B) (Rule 4)

4. <expr>.actual_type ← either int or real (Rule 2)

5. <expr>.expected_type == <expr>.actual_type is
   either TRUE or FALSE (Rule 2)

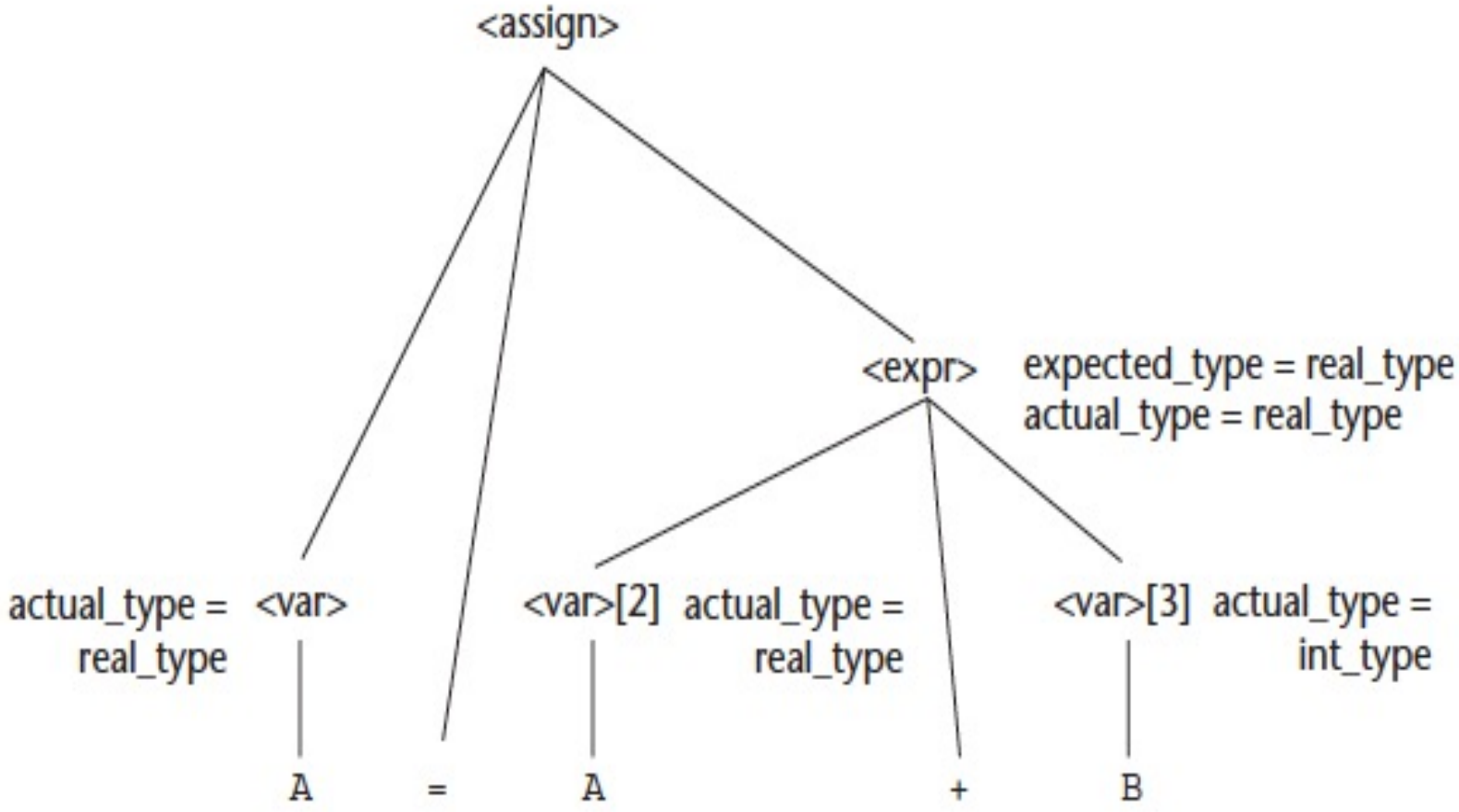
Computing Attribute Values

- --- show attribute flow in the tree



Computing Attribute Values

- Fully attributed parse tree (A is real, B is int)



Dynamic Semantics

- There is no single widely acceptable notation or formalism for describing semantics
- Several needs for a methodology and notation for semantics:
  - Programmers need to know what statements mean
  - Compiler writers must know exactly what language constructs do
  - Compiler generators would be possible
  - Designers could detect ambiguities and inconsistencies

1. Operational Semantics

- Describe the meaning of a program by executing its statements on a machine, either simulated or actual.
- The change in the state of the machine (memory, registers, etc.) defines the meaning of the statement
- To use operational semantics for a high-level language, a virtual machine (interpreter) is needed

2. Denotational Semantics

- Based on recursive function theory
- The most abstract semantics description method
- Programming language constructs are mapped to mathematical objects (e.g. sets or functions)

Denotational Semantics for Decimal Numbers

- Syntax rule:

<dec_num> → '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9' |
<dec_num> ('0' | '1' | '2' | '3' | '4' | '5' | '6' |
'7' | '8' | '9')

- Denotational mappings for the rule:

M_{dec}('0') = 0, M_{dec} ('1') = 1, …, M_{dec} ('9') = 9

M_{dec} (<dec_num> '0') = 10 * M_{dec} (<dec_num>)

M_{dec} (<dec_num> '1') = 10 * M_{dec} (<dec_num>) + 1

…

M_{dec} (<dec_num> '9') = 10 * M_{dec} (<dec_num>) + 9

Denotational Semantics: program state

- The state of a program is the values of all its current variables

$$s = \{\langle i_1, v_1\rangle, \langle i_2, v_2\rangle, \ldots, \langle i_n, v_n\rangle\}$$

i_j → variable name

v_j → variable i_j's current value

*v_j = **undef**, if i_j is currently undefined*

- Let **VARMAP** be a function that, when given a variable name and a state, returns the current value of the variable

$$\text{VARMAP}(i_j, s) = v_j$$

Denotational Semantics: Expressions

- We assume simple expressions:
  - One arithmetic operator (+ or *)
  - Integer variables or integer literals as operands
  - No parenthesis
- BNF:

```
<expr> → <dec_num> | <var> | <bin_expr>
<bin_expr> → <left_expr> <operator> <right_expr>
<left_expr> → <dec_num> | <var>
<right_expr> → <dec_num> | <var>
<operator> → + | *
```

Denotational Semantics: Expressions

```
Me(<expr>, s) Δ =
    case <expr> of
        <dec_num> => Mdec(<dec_num>, s)

        <var> => if VARMAP(<var>, s) == undef
                   then error
                   else VARMAP(<var>, s)

        <binary_expr> =>
                 if (Me(<binary_expr>.<left_expr>,s) == undef OR
                     Me(<binary_expr>.<right_expr>,s) == undef)
                 then error
                 else if (<binary_expr>.<operator> == '+'
                    then Me(<binary_expr>.<left_expr>, s) +
                           Me(<binary_expr>.<right_expr>, s)
                    else Me(<binary_expr>.<left_expr>, s) *
                           Me(<binary_expr>.<right_expr>, s)
```

- Note: "Δ=" is used to define mathematical functions

Denotational Semantics vs Operational Semantics

- In operational semantics, the state changes are defined by coded algorithms
- In denotational semantics, the state changes are defined by rigorous mathematical functions

Axiomatic Semantics

- Based on formal logic (predicate calculus)
- Original purpose: formal program verification
- Axioms or inference rules are defined for each statement type in the language (to allow transformations of logic expressions into more formal logic expressions)
- The logic expressions are called *assertions*

Summary

- BNF and context-free grammars are equivalent meta-languages
  - Well-suited for describing the syntax of programming languages
- An attribute grammar is a descriptive formalism that can describe both the syntax and the semantics of a language
- Three primary methods of semantics description
  - Operation, axiomatic, denotational